Reflexion zum Aufbau eines Modul Assistenten

Lernstand vom 4. November 2026

Dieser Bericht hält den bisher im Projekt und im Gespräch dokumentierten Lernweg fest. Die technischen Vorbereitungen und Codeüberarbeitungen entstanden mit KI-Unterstützung. Eigene Fragen, Entscheidungen und ausgeführte Schritte werden deshalb von vorbereiteten Beispielen und noch ausstehenden Versuchen getrennt. Der Bericht beschreibt einen Zwischenstand; die vollständige eigene Bearbeitung der Notebooks 02 bis 04 und der persönliche Promptvergleich stehen noch aus.

Technisches Verständnis

Zu Beginn beschäftigte ich mich zunächst mit der Einrichtung: Warum sollte Python 3.12 nötig sein, welche Rolle spielt die Datei requirements.txt, und wozu dient setup.bat? Ich prüfte die Python-Version in meiner Projektumgebung selbst und erhielt Python 3.13.15. Anschliessend startete ich JupyterLab und öffnete das erste Notebook. Diese Schritte machten die Einrichtung für mich konkreter. Die getrennte Umgebung .venv und die darin installierten Pakete bilden die Grundlage dafür, dass die Beispiele auf meinem Rechner ausgeführt werden können.

Beim Lesen des ersten Notebooks fiel mir auf, dass zu viele neue Begriffe gleichzeitig vorkamen. Ich bat deshalb um kürzere Codeabschnitte und Erklärungen direkt vor dem jeweiligen Schritt. Auch die Ich-Form passte für mich nicht zu einer allgemeinen Anleitung. Diese Rückmeldung führte zu einer klareren Trennung von erklärendem Text, ausführbarem Code und persönlicher Reflexion. Entscheidend ist für meinen weiteren Lernweg, jeden Schritt erklären zu können, statt ein vollständig vorbereitetes Notebook lediglich auszuführen.

Eine weitere Frage betraf ChatOpenAI: Der Name erweckte bei mir den Eindruck, dass eine andere Schnittstelle als die BFH verwendet würde. Im Projekt dient diese Klasse jedoch als LangChain-Adapter für die kompatible BFH-Schnittstelle. Modellname, Serveradresse und Adapter sind unterschiedliche Teile der Verbindung. Die folgenden Notebooks vertiefen weitere Unterschiede: Ein Gesprächsverlauf stellt frühere Nachrichten bereit, RAG sucht passende Dokumentstellen, und ein Agent wählt zwischen Werkzeugen. Die eigene Anwendung dieser Konzepte muss noch durch meine Versuche ergänzt werden.

Zusammenarbeit mit KI

Die KI unterstützte beim Aufbau des Repositorys, bei Code, Dokumentation und technischen Prüfungen. Meine Beiträge bestanden unter anderem darin, die Verständlichkeit zu hinterfragen, Anforderungen mit der Aufgabenstellung abzugleichen und die Eingabedokumente auszuwählen. Ich schlug echte kurze Modulunterlagen vor und stellte die Zusammenfassungen zu Bitcoin und Open Banking bereit. Dadurch beziehen sich die Beispiele auf Inhalte meines Moduls.

Das Prompt-Logbuch war zunächst unklar: Ich wusste nicht, welcher Prompt damit gemeint war. Die Erklärung unterscheidet nun den Arbeitsauftrag an das Modell von dessen Antwort. Ein genauerer Arbeitsauftrag kann Sprache, Antwortformat, Quellenangabe und den Umgang mit fehlenden Informationen vorgeben. Die eingefügten Beispiele stammen von der KI und sind als solche gekennzeichnet. Sie belegen noch keinen eigenen Vergleich. Für diesen Nachweis werde ich einen eigenen Erstprompt, die tatsächliche Überarbeitung und beide erhaltenen Antworten dokumentieren.

Herausforderungen und Fehlersuche

Nach dem Eintragen meines BFH-Schlüssels trat beim Modellaufruf ein Fehler mit HTTP-Status 404 auf. Ich teilte die Fehlermeldung im Gespräch. Die anschliessende technische Prüfung durch die KI zeigte, dass der bisher verwendete Proxy-Pfad falsch war. Nach der Korrektur auf /v1 funktionierte der kurze Verbindungstest; anschliessend meldete ich ebenfalls zurück, dass der Aufruf zu funktionieren schien.

Dieser Fall zeigt, warum die Bezeichnung einer Fehlermeldung allein nicht genügt. Ein als Modellfehler dargestelltes Problem kann an der Adresse der Schnittstelle liegen. Für weitere Fehler möchte ich deshalb zuerst den betroffenen Schritt eingrenzen: Einrichtung, Verbindung, Suche oder Verarbeitung der Antwort. Der erfolgreiche Verbindungstest ersetzt dabei keine vollständige Prüfung der Antworten des Assistenten.

Entwicklung der Projektidee

Mit der Auswahl der beiden PDFs wurde die Idee eines Modul-Assistenten konkreter. Testpersonen sollen dieselben Unterlagen erhalten und damit dieselbe Wissensgrundlage nutzen können. Trotzdem können Modellantworten unterschiedlich ausfallen. Deshalb sind nachvollziehbare Fragen, überprüfbare Quellenangaben und ein offener Umgang mit fehlenden Informationen wichtig.

Eine überzeugend formulierte Antwort reicht als Qualitätsnachweis nicht aus. Die angegebene Seite muss die Aussage tatsächlich belegen. Ebenso unterscheidet sich ein vorbereiteter Demo-Ablauf von einer echten Modellentscheidung. Beim eigenen Durcharbeiten werde ich deshalb besonders auf die gefundenen Textstellen, die übergebenen Eingaben und die tatsächlichen Werkzeugergebnisse achten. Erst diese eigenen Beobachtungen erlauben eine abschliessende persönliche Bewertung des fertigen Assistenten.

Zwei mögliche Erweiterungen

Grafiken erschliessen. Die Open-Banking-Unterlage enthält Darstellungen, die beim einfachen Auslesen des PDF-Textes teilweise fehlen. Eine Erweiterung mit Texterkennung oder geeigneter Bildverarbeitung könnte diese Inhalte zugänglich machen. Die erkannten Informationen müssten anhand der Originalseite geprüft und weiterhin mit einer Quelle verbunden werden.

Systematischere Qualitätsprüfung. Ein grösserer, vorab festgelegter Fragenkatalog könnte beantwortbare Fragen, unbeantwortbare Fragen und mehrdeutige Folgefragen enthalten. Wiederholte Durchläufe würden zeigen, wie stabil Antworten, Quellen und Werkzeugwahl sind. Eine manuelle Belegprüfung bliebe nötig, weil einfache Wortvergleiche fachliche Richtigkeit nicht zuverlässig erfassen.